PHASE 6 OF 18

OOP I: Encapsulation, Packages & Inheritance

Encapsulation and JavaBeans, access modifiers, non-access modifiers, packages and imports, inheritance types, constructor chaining, super, and IS-A vs HAS-A

IN THIS PHASE

6.1 Encapsulation
6.2 Access Modifiers
6.3 Non-Access Modifiers at a Glance
6.4 Packages
6.5 Inheritance
6.6 Constructors and Inheritance (Constructor Chaining)
6.7 The super Keyword
6.8 IS-A vs HAS-A (Inheritance vs Composition)
6.9 Predict the Output
6.10 Interview Questions
6.11 Quick Revision

6.1 Encapsulation

Encapsulation means bundling data (fields) and the methods that operate on it into one unit (a class), and **restricting direct access** to that data. A Java class is the textbook example.

The recipe:

1. Make fields `private`, so no outside code can read or change them directly.
2. Provide `public` **getter** methods to read and **setter** methods to update, and only where needed.
3. Put **validation and business rules** inside the setters and other methods.

```
public class BankAccount {
    private final String accountNo;     // read-only after construction: no setter
    private String holder;
    private double balance;

    public BankAccount(String accountNo, String holder) {
        this.accountNo = accountNo;
        setHolder(holder);              // reuse validation
    }

    public String getAccountNo() { return accountNo; }
    public String getHolder()    { return holder; }
    public double getBalance()   { return balance; }

    public void setHolder(String holder) {
        if (holder == null || holder.isBlank())
            throw new IllegalArgumentException("Holder name required");
        this.holder = holder.trim();
    }

    public void deposit(double amount) {         // behaviour instead of setBalance()
        if (amount <= 0) throw new IllegalArgumentException("Amount must be positive");
        balance += amount;
    }

    public void withdraw(double amount) {
        if (amount > balance) throw new IllegalStateException("Insufficient funds");
        balance -= amount;
    }
}
```

Without vs with encapsulation

```
class Student { public int id; }          // WITHOUT
Student s = new Student();
s.id = 101;   s.id = -500;   s.id = 99999;   // anything goes, no control

class SafeStudent {                        // WITH
    private int id;
    public void setId(int id) {
        if (id <= 0) throw new IllegalArgumentException("id must be positive");
        this.id = id;
    }
    public int getId() { return id; }
}
SafeStudent t = new SafeStudent();
// t.id = 101;          // COMPILE ERROR: id has private access
t.setId(101);           // the only way in, and it's validated
```

INTERVIEW

"If a setter can change the field, how is that encapsulation?" Encapsulation does not forbid change. It forbids **direct, uncontrolled** change. Every change goes through code the class owns, where it can be validated, logged, made thread-safe, or reimplemented later without breaking callers.

Benefits

Benefit	Example
Data hiding / security	`balance` can't be set to a random value from outside
Validation and control	Reject negative amounts, blank names, invalid ages
Flexibility to change internals	Switch `double balance` to `BigDecimal` without touching callers
Read-only or write-only properties	Getter without setter (`accountNo`)
Easier testing and maintenance	Behaviour lives in one place

Encapsulation vs data hiding vs abstraction

Concept	Focus	How
Data hiding	Keep data inaccessible from outside	`private`
Encapsulation	Data + methods together, with controlled access	Class + private fields + public methods
Abstraction	Expose **what** an object does, hide **how**	Interfaces, abstract classes (Phase 7)

JavaBeans, POJOs and boilerplate

- **POJO** (Plain Old Java Object): an ordinary class with no framework requirements.
- **JavaBean** conventions: private fields, a public no-arg constructor, `getX()`/`setX()` (`isX()` for booleans), and often `Serializable`. Frameworks such as Spring, Hibernate/JPA and Jackson rely on these names.

> **MODERN JAVA**
> Two ways to cut getter/setter boilerplate:
> - **Lombok** annotations: `@Getter`, `@Setter`, `@NoArgsConstructor`, `@AllArgsConstructor`, `@Data`, `@Builder`.
> - **Records** (Java 16+): `public record Customer(String id, String name) { }` gives private final fields, a constructor, accessors (`id()`, `name()`), `equals`, `hashCode` and `toString` in one line. Records are covered in Phase 15.

6.2 Access Modifiers

Access modifiers control **visibility** of classes, fields, methods and constructors. Java has four levels, from most to least open:

Modifier	Same class	Same package	Subclass (other package)	Anywhere (other package)
`public`	Yes	Yes	Yes	Yes
`protected`	Yes	Yes	Yes (through inheritance)	No
default (no keyword, "package-private")	Yes	Yes	No	No
`private`	Yes	No	No	No

Where each can be used

Element	`public`	`protected`	**default**	`private`
Top-level class / interface / enum / record	Yes	**No**	Yes	**No**
Nested (member) class	Yes	Yes	Yes	Yes
Field	Yes	Yes	Yes	Yes
Method	Yes	Yes	Yes	Yes
Constructor	Yes	Yes	Yes	Yes (singleton, utility, factory)
Local variable	No access modifiers allowed (only `final`)			

```java
package com.bank.core;

public class Employee {
    public    void m1() { }   // visible everywhere
    protected void m3() { }   // package + subclasses
              void m4() { }   // package only (default)
    private   void m2() { }   // this class only

    public static void main(String[] args) {
        Employee emp = new Employee();
        emp.m1(); emp.m2(); emp.m3(); emp.m4();   // all OK inside the same class
    }
}
```

The protected subtlety

A subclass in a **different package** can use an inherited protected member only **through its own type** (`this` or a subclass reference), not through a plain parent reference.

```java
package com.bank.core;
public class Account { protected double balance; }

package com.bank.loans;
import com.bank.core.Account;

public class LoanAccount extends Account {
    void test(Account other, LoanAccount mine) {
        this.balance = 100;      // OK: inherited through own type
        mine.balance = 200;      // OK: subclass reference
        // other.balance = 300;  // COMPILE ERROR: not via a plain Account reference
    }
}
```

Rules worth remembering

1. One `.java` file can contain many top-level classes, but **at most one** `public` class, and the file must be named after it.
2. A top-level class can only be `public` or default.
3. Overriding methods **cannot reduce visibility**: a `public` method can't be overridden as `protected` (Phase 7).
4. Choose the **most restrictive** level that works: fields `private`, helper methods `private`, API methods `public`.

> **NOTE**
> Reflection (`field.setAccessible(true)`) can bypass `private`. Since Java 9, the **module system** blocks this for packages that a module doesn't open. That is why some old libraries print "illegal reflective access" warnings on newer JDKs.

6.3 Non-Access Modifiers at a Glance

Modifier	Class	Method	Field	Meaning (covered in)
`static`	nested only	Yes	Yes	Belongs to the class (Phase 5)
`final`	Yes	Yes	Yes	Can't be extended / overridden / reassigned (Phase 7)
`abstract`	Yes	Yes	No	Incomplete; must be extended / implemented (Phase 7)
`synchronized`	No	Yes (and blocks)	No	One thread at a time per lock (Phase 11)
`volatile`	No	No	Yes	Visibility of updates across threads (Phase 11)
`transient`	No	No	Yes	Skipped during serialization (Phase 12)
`native`	No	Yes	No	Implemented in C/C++ via JNI (Phase 1)
`strictfp`	Yes	Yes	No	Strict IEEE-754 floating point. **Obsolete since Java 17**.
`sealed` / `non-sealed`	Yes	No	No	Restricts which classes may extend (Java 17, Phase 15)

CORRECTED FROM ORIGINAL NOTES

The short notes present `strictfp` as useful for "platform-independent results". Since **Java 17** (JEP 306) all floating-point arithmetic is strict by default, so `strictfp` has no effect and the compiler warns that it's redundant.

REMEMBER

Illegal combinations: `abstract final`, `abstract private`, `abstract static` (methods). A method can't be both abstract and have a body.

6.4 Packages

A **package** is a namespace that groups related classes, interfaces, enums, records and sub-packages, much like folders on disk.

Why packages?

- **Organisation:** group by feature or layer (`controller`, `service`, `repository`).
- **Name-clash avoidance:** `java.util.Date` and `java.sql.Date` can coexist.
- **Access control:** default and `protected` visibility are package-based.
- **Reuse and distribution:** package as JARs.

Built-in packages

Package	Contains
`java.lang`	`String`, `Object`, `Math`, wrappers, `System`, `Thread`, exceptions. **Imported automatically.**
`java.util`	Collections, `Scanner`, `Optional`, `Random`, `UUID`
`java.util.concurrent`	Executors, concurrent collections, locks, atomics
`java.io` / `java.nio.file`	Streams, readers, writers, `Files`, `Path`
`java.time`	`LocalDate`, `LocalDateTime`, `Duration`, `ZonedDateTime`
`java.math`	`BigDecimal`, `BigInteger`
`java.net` / `java.net.http`	Networking, URLs, `HttpClient`
`java.sql`	JDBC: `Connection`, `PreparedStatement`, `ResultSet`
`java.text`	`NumberFormat`, `SimpleDateFormat` (legacy)

Creating and using a package

```
// File: src/com/bank/payments/PaymentService.java
package com.bank.payments;          // must be the FIRST statement (comments may precede it)

import java.util.List;              // single-type import (preferred)
import java.util.*;                 // on-demand (wildcard) import
import static java.lang.Math.max;   // static import of one member
import static java.lang.Math.*;     // static import of all static members

public class PaymentService { }
```

```
src/
 └── com/bank/payments/PaymentService.java       folder structure mirrors the package

javac -d out src/com/bank/payments/PaymentService.java   # -d creates package folders
java -cp out com.bank.payments.PaymentService            # run by FULLY QUALIFIED name
```

Naming convention

All lowercase, using the reverse internet domain plus project and module: `com.companyname.projectname.module`, for example `com.icici.netbanking.transfer` or `org.apache.commons.lang3`.

Import rules

1. `java.lang` is always imported implicitly.
2. A wildcard import (`java.util.*`) does **not** import sub-packages: `java.util.function.Function` needs its own import.
3. Imports only save typing. They don't load or copy code; the bytecode always uses fully qualified names. (Unlike C's `#include`, nothing is copied.)
4. When two imported packages have the same simple name, use the **fully qualified name** for at least one.
5. Classes in the **default (unnamed) package** can't be imported by classes in named packages, so always use packages in real code.

```
import java.util.Date;
public class Report {
    Date utilDate = new Date();                          // java.util.Date
    java.sql.Date sqlDate = java.sql.Date.valueOf("2026-01-31");  // fully qualified
}
```

TIP

In Eclipse, **Ctrl + Shift + O** organises imports (adds missing ones, removes unused ones). The IntelliJ equivalent is **Ctrl + Alt + O**.

MODERN JAVA

Java 9 modules (`module-info.java`) group packages and declare what a module `requires` and `exports`. Unexported packages are hidden even if their classes are public. Most applications still use the classpath, but the JDK itself is modular (`java.base`, `java.sql`, ...).

6.5 Inheritance

Inheritance lets a class (the **child**: subclass or derived class) acquire the fields and methods of another class (the **parent**: superclass or base class) using `extends`. It models an **IS-A** relationship: a `SavingsAccount` IS-A `Account`.

```
class Account {                                  // parent / superclass / base class
    protected String accountNo;
    protected double balance;
    void deposit(double amt) { balance += amt; }
    String summary() { return accountNo + " : " + balance; }
}

class SavingsAccount extends Account {           // child / subclass / derived class
    private double interestRate = 4.0;           // child adds new state
    void addInterest() {                         // ...and new behaviour
        deposit(balance * interestRate / 100);   // reuses the inherited method
    }
}

SavingsAccount sa = new SavingsAccount();
sa.accountNo = "SB-1";      // inherited field
sa.deposit(1000);           // inherited method
sa.addInterest();           // own method
System.out.println(sa.summary());   // SB-1 : 1040.0
```

Advantages: code reuse, less duplication, easier maintenance, a natural type hierarchy, and it enables **runtime polymorphism** (Phase 7).

What is (and isn't) inherited

Member	Inherited?
`public` / `protected` fields and methods	Yes
Default (package-private) members	Only if the child is in the **same package**
`private` members	**No direct access**. They exist inside the child object but are reachable only through inherited public/protected methods.
Constructors	**No**. They are invoked through `super(...)`, never inherited.
Static members	Accessible through the child, but **hidden** rather than overridden
Instance/static initializer blocks	No (they run as part of the parent's construction)

Reference type vs object type

```
class Parent { void m1() { System.out.println("Parent m1"); } }
class Child extends Parent {
    @Override void m1() { System.out.println("Child m1"); }
    void m2() { System.out.println("Child m2"); }
}

Parent p = new Child();   // parent reference, child object (upcasting: automatic)
p.m1();                   // Child m1  -> the OBJECT type decides which override runs
// p.m2();                // COMPILE ERROR -> the REFERENCE type decides what is visible

Child c = new Child();
c.m1();  c.m2();          // Child m1, Child m2

// Child bad = new Parent();   // COMPILE ERROR: a Parent is not necessarily a Child
```

> **REMEMBER**
> The **compiler** checks member access using the **reference type**. The **JVM** picks the overridden method using the **object type** at runtime (dynamic dispatch). Method lookup starts in the object's own class, then moves up the parents to `Object`.

Types of inheritance

```
 Single          Multilevel       Hierarchical          Multiple (classes: NOT allowed)
   A                 A                 A                    A       B
   ^                 ^               / | \                   \     /
   B                 B              B  C  D                   \   /
                     ^                                          C
                     C                      Hybrid = mix including multiple: NOT via classes
```

Type	Supported with classes?	Example
Single	Yes	`class B extends A`
Multilevel	Yes	`A ← B ← C` (`Object ← Account ← SavingsAccount`)
Hierarchical	Yes	`Savings`, `Current`, `FixedDeposit` all extend `Account`
Multiple	**No** (for classes)	`class C extends A, B` is a compile error
Hybrid	**No** (for classes)	A mix that includes multiple inheritance

Multiple inheritance **of type** (and of default behaviour) **is** possible with **interfaces**: `class C implements A, B` (Phase 7).

Why no multiple inheritance of classes? The diamond problem

```
class A { void show() { System.out.println("A"); } }
class B extends A { void show() { System.out.println("B"); } }
class C extends A { void show() { System.out.println("C"); } }
// class D extends B, C { }     // INVALID. new D().show() -> B's or C's version?
```

If one class could extend two parents that both define `show()` (or both declare a field `id`), the call would be **ambiguous**. Java avoids the problem for classes; for interfaces with conflicting default methods, it forces you to override and choose (Phase 7).

> **CORRECTED FROM ORIGINAL NOTES**
> The original notes say the lack of multiple inheritance "makes Java a partial OOP language". That is not the reason. Java is called not *purely* object-oriented because of **primitive types** and **static members**. Lack of multiple class inheritance is a deliberate design choice.

Cyclic inheritance is illegal

```
// class User extends User { }                                  // COMPILE ERROR
// class A extends B { }  class B extends A { }                 // COMPILE ERROR: cyclic
```

> **NOTE**
> Every class has exactly one direct superclass. If none is written, it implicitly `extends Object`. So `java.lang.Object` is the root of every hierarchy, and all classes can use its 11 methods: `toString`, `equals`, `hashCode`, `getClass`, `clone`, `finalize`, `wait` (×3), `notify` and `notifyAll`.

6.6 Constructors and Inheritance (Constructor Chaining)

Creating a child object **always runs the parent's constructor first**, all the way up to `Object`. The parent part of the object must be initialised before the child part.

```
class User {
    User() { System.out.println("Parent Constructor"); }
}
class Employee extends User {
    Employee() {
        // super();   <- inserted automatically by the compiler if you don't write it
        System.out.println("Child Constructor");
    }
}
new Employee();
// Parent Constructor
// Child Constructor
```

> **CORRECTED FROM ORIGINAL NOTES**
> The original notes say "if we remove `super()`, the child class will not compile". In fact, if a constructor doesn't start with `this(...)` or `super(...)`, the compiler **inserts** `super()` automatically. Compilation fails only when the parent has **no no-arg constructor**, as shown below.

```
class User {
    User(int id, String name) { System.out.println("User(" + id + ", " + name + ")"); }
}
class Employee extends User {
    Employee() {
        super(101, "Raju");           // REQUIRED: User has no no-arg constructor
        System.out.println("Employee()");
    }
}
// Without the super(...) line -> COMPILE ERROR:
//   constructor User in class User cannot be applied to given types
```

Rules for super() and this()

1. `super(...)` calls a parent constructor; `this(...)` calls another constructor of the same class.
2. Either must be the **first statement** in the constructor (Java 25 relaxes this slightly with *flexible constructor bodies*: statements that don't touch `this` may come first).
3. You can't have both `this(...)` and `super(...)` in the same constructor, but a `this(...)` chain eventually reaches a `super(...)`.
4. They can be used only inside constructors.

Full order with static and instance blocks (verified on JDK 21)

```
class Parent {
    static { System.out.println("1. Parent static block"); }
    { System.out.println("3. Parent instance block"); }
    Parent() { System.out.println("4. Parent constructor"); }
}
class Child extends Parent {
    static { System.out.println("2. Child static block"); }
    { System.out.println("5. Child instance block"); }
    Child() { System.out.println("6. Child constructor"); }
}
new Child();      // prints 1 to 6 in order
new Child();      // prints only 3, 4, 5, 6 (static blocks ran once)
```

6.7 The super Keyword

Use	Syntax	Purpose
Parent constructor	`super(args)`	Initialise the parent part (first statement)
Parent method	`super.method()`	Call the parent's version from an override, to extend rather than replace it
Parent field	`super.field`	Access a parent field hidden by a child field with the same name

```
class Parent {
    int x = 10;
    void m1() { System.out.println("Parent m1()"); }
}
class Child extends Parent {
    int x = 20;                                  // HIDES Parent.x (avoid in real code)
    @Override
    void m1() {
        super.m1();                              // reuse the parent behaviour first
        System.out.println("Child m1()");
    }
    void display() {
        System.out.println("Parent x = " + super.x);   // 10
        System.out.println("Child x = " + this.x);     // 20
    }
}
```

PITFALL

Fields are not polymorphic. With `Parent p = new Child();`, `p.x` gives **10** (decided by the reference type), while `p.m1()` runs Child's override. Keep fields `private` and never redeclare a parent's field name in a child.

`this` vs `super`:

this	super
Refers to the current object	Refers to the parent part of the current object
`this(...)` calls a constructor of the same class	`super(...)` calls a parent constructor
`this.field` / `this.method()`	`super.field` / `super.method()`
Not usable in static context	Not usable in static context

6.8 IS-A vs HAS-A (Inheritance vs Composition)

Relationship	Meaning	Java mechanism	Example
IS-A	Specialisation	`extends` / `implements`	`SavingsAccount` IS-A `Account`
HAS-A (association)	One object uses or holds another	A field referencing another object	`Customer` HAS-A `Address`
Aggregation (weak HAS-A)	The part can exist without the whole	Object passed in from outside	`Department` has `Employee`s (employees outlive the department)
Composition (strong HAS-A)	The part's lifecycle is owned by the whole	Object created inside, not shared	`Car` has an `Engine` created with it

```
class Engine { void start() { System.out.println("Engine started"); } }

class Car {                                     // composition: Car HAS-A Engine
    private final Engine engine = new Engine();
    void drive() { engine.start(); System.out.println("Driving"); }
}

class Department {                              // aggregation: employees supplied from outside
    private final List<Employee> staff;
    Department(List<Employee> staff) { this.staff = staff; }
}
```

TIP

Favour composition over inheritance. Use `extends` only for a true IS-A relationship where the child can be used anywhere the parent is expected (the Liskov Substitution Principle). Composition is more flexible: you can swap parts at runtime and avoid fragile deep hierarchies. Frameworks like Spring wire HAS-A dependencies through dependency injection.

6.9 Predict the Output

```
class P { P() { System.out.print("P "); } }
class Q extends P { Q() { System.out.print("Q "); } }
class R extends Q { R() { super(); System.out.print("R "); } }
new R();                                        // P Q R

class X { int v = 1; int get() { return v; } }
class Y extends X { int v = 2; int get() { return v; } }
X obj = new Y();
System.out.println(obj.v + " " + obj.get());  // 1 2  (fields: reference type; methods: object)

class M { M(int a) { } }
// class N extends M { }       // COMPILE ERROR: implicit super() finds no M()

class S { static void hi() { System.out.println("S"); } }
class T extends S { static void hi() { System.out.println("T"); } }
S s = new T();
s.hi();                                         // S  (static methods are hidden, not overridden)
```

6.10 Interview Questions

Q. What is encapsulation and how is it achieved?

Bundling data with the methods that operate on it and restricting direct access. Achieved by `private` fields plus public methods (getters/setters or behaviour methods) that enforce validation and invariants.

Q. What is the difference between encapsulation and abstraction?
Encapsulation hides **data** and controls access to it (the how of protection). Abstraction hides **implementation complexity** behind a simple interface (the what an object does). Encapsulation is implemented with access modifiers; abstraction with interfaces and abstract classes.

Q. Explain the four access modifiers.
`private`: same class only. Default: same package. `protected`: same package plus subclasses in other packages (through inheritance). `public`: everywhere. Top-level classes can only be public or default.

Q. Can a top-level class be private or protected?
No. Only nested classes can use those modifiers.

Q. What is the use of packages?
To organise related types, avoid naming conflicts, control access (package-private and protected) and support reuse and distribution as JARs.

Q. Does `import java.util.*` import `java.util.function.*`?
No. Wildcard imports cover a single package only, not sub-packages.

Q. Why doesn't Java support multiple inheritance with classes?
To avoid the diamond problem: ambiguity when two parents provide the same method or field. Java allows multiple inheritance of type through interfaces, and resolves default-method conflicts by forcing an override.

Q. Are constructors inherited? Are private members inherited?
Constructors are never inherited; they are chained via `super(...)`. Private members are not accessible in the subclass, although they exist inside the object and can be reached through inherited non-private methods.

Q. In what order are constructors called in a hierarchy?
Top-down: `Object` first, then each parent, then the child. Each constructor's first action is `super(...)` (explicit or implicit).

Q. What happens if the parent class has only a parameterized constructor?
Every child constructor must explicitly call `super(args)`. Otherwise the implicit `super()` fails to compile.

Q. What is the difference between `this` and `super`?
`this` refers to the current object (its fields, methods, other constructors). `super` refers to the parent part (parent constructor, overridden methods, hidden fields). Neither is allowed in a static context.

Q. What is the difference between IS-A and HAS-A? Which should you prefer?
IS-A is inheritance (`extends`/`implements`); HAS-A is composition or aggregation (an object holds a reference to another). Prefer composition unless a genuine substitutable IS-A relationship exists.

6.11 Quick Revision

- **Encapsulation** = private fields + controlled public methods (validation inside). It controls change; it doesn't forbid it.
- Access levels: `private` < default (package) < `protected` (package + subclasses) < `public`.
- Top-level types are only `public` or default; one public class per file.
- `strictfp` is obsolete since Java 17; `abstract` can't combine with `final`, `private` or `static`.
- Packages: lowercase reverse domain; `package` is the first statement; `java.lang` is auto-imported; wildcards skip sub-packages; use FQNs for clashes.
- `extends` = IS-A. Inherited: public/protected (+ default in the same package). Not inherited: constructors, private access.
- Class inheritance types: single, multilevel, hierarchical. Multiple and hybrid only via interfaces.
- The compiler checks the **reference** type; the JVM runs the **object** type's override. Fields and static methods use the reference type.

- Parent constructors run first; `super()` is implicit unless the parent lacks a no-arg constructor.
- `this(...)` / `super(...)`: first statement, never both.
- Order: parent static → child static → parent instance blocks + constructor → child instance blocks + constructor.
- Prefer **composition (HAS-A)** over inheritance unless IS-A truly holds.